

Welcome to Parallel & Distributed Computing

Assignment #1

A Revision of Multithreading (Pthreads) & Performance Evaluation

Spring 2026

Instructions

- For your assignment, rename all your code files according to the following format e.g. ROLL-NUM-SECTION-TASK#-VERSION.EXT (23i-0001-A-TASK1-V1.c) and so on. Each file that you submit must contain your name, student-id, and assignment# on top of the file in comments.
- Combine all your work in one folder. The folder must contain only code and pdf file (no binaries, no exe files etc.).
- Rename the folder as ROLL-NUM-SECTION (e.g. 23i-0001-A) and compress the folder as a zip file (e.g. 23i-0001-A.zip). Do not submit .rar file.
- All the submissions will be done on Google classroom within the deadline. Submissions other than Google classroom (e.g. email etc.) will not be accepted.
- The student is solely responsible to check the final zip files for issues like corrupt files, viruses in the file, mistakenly exe sent. If we cannot download the file from Google classroom due to any reason, it will lead to zero marks in the assignment.
- If there is a syntax error in code, zero marks will be awarded in that part of assignment.
- Displayed output should be well-formatted and well presented. Use appropriate comments and indentation in your source code.
- Understanding the assignment is part of the evaluation.
- Be prepared for viva or any other evaluation mechanism after the submission of assignment.
- *Late Submission & Plagiarism* policies will be applied as described in course outline.

Tip: For timely completion of the assignment, start as early as possible.

Objective

Before diving deep into the complexities of parallel and distributed systems, it's crucial to understand why this field is important and how it can be applied effectively. Keeping in view that you already know multi-threading, your first assignment will be an introductory exercise using multi-threading that demonstrates the power of parallelism and the benefits of multi-threading in computational tasks.

You are assigned three tasks, a little different in nature/characteristics, and you are required to implement and evaluate all three tasks using three approaches (we will call them versions):

- **V1: Serial Baseline** Begin by implementing the task in a single-threaded manner, where the operations are executed in a straightforward, step-by-step sequence. This serves as the baseline for understanding the complexity of the task.
- **V2: Multi-threaded Execution (default OS scheduling)** Next, parallelize the task by utilizing multiple threads. In this version, the operating system will schedule threads across CPU cores (no manual affinity setting).

- **V3: Multi-threaded Execution (User-Assigned Affinity and Scheduling)** Finally, implement the task again using multiple threads, but this time manually assign each thread to a specific CPU core using `pthread_setaffinity_np()` and implement a clear scheduling strategy beyond V2.

You will perform performance evaluation (runtime, speedup, efficiency) across multiple thread counts and submit a report.

Datasets

Three datasets are attached on GCR along with this assignment, the links to the datasets are provided for details about the datasets:

1. SUSY: <https://archive.ics.uci.edu/dataset/279/susy>
2. SuiteSparse-webbase-1M: <https://sparse.tamu.edu/Williams/webbase-1M>
3. SNAP-web-Google: <https://snap.stanford.edu/data/web-Google.html>

Tasks

The detailed dataset and task description for each task is provided below.

Task 1: Batch Scoring with a Trained Classifier (Logistic Regression)

For this task you will use SUSY dataset. SUSY is a large CSV dataset where each row is one record labeled as either **signal (1)** or **background (0)**, followed by **18 numeric features**. Each record has the format:

$$y, x_1, x_2, \dots, x_{18} \quad \text{where } y \in \{0, 1\}$$

Task Description

Apply the fixed logistic regression model to every record and summarize:

- Linear score:

$$s = b + \sum_{i=1}^{18} w_i x_i$$

- Probability:

$$p = \frac{1}{1 + e^{-s}}$$

- Prediction: $\hat{y} = 1$ if $p \geq 0.5$, else $\hat{y} = 0$
- Accumulate totals across all records:
 - PRED_POS: total number of records predicted as positive (signal), i.e., the count of records where $\hat{y} = 1$.
 - TP (True Positives): number of records where the true label is positive and the prediction is positive, i.e., $y = 1$ and $\hat{y} = 1$.
 - FP (False Positives): number of records where the true label is negative but the prediction is positive, i.e., $y = 0$ and $\hat{y} = 1$.
 - TN (True Negatives): number of records where the true label is negative and the prediction is negative, i.e., $y = 0$ and $\hat{y} = 0$.
 - FN (False Negatives): number of records where the true label is positive but the prediction is negative, i.e., $y = 1$ and $\hat{y} = 0$.

Fixed Constants

Bias: $b = -0.35$

Weights $w[1..18]$ corresponding to features $x1..x18$:

[0.12, -0.07, 0.05, 0.09, -0.11, 0.03, 0.08, -0.02,
0.06, 0.04, -0.05, 0.10, -0.08, 0.07, 0.02, -0.03,
0.11, -0.06]

Example

Example record (same schema; y followed by $x1..x18$):

1.0, 0.90, -0.50, 0.30, 0.80, -0.70, 0.60, 0.70, 0.10,
0.40, 0.50, -0.60, 0.90, -0.40, 0.50, 0.20, 0.10,
0.80, -0.50

Using $b = -0.35$ and the provided weights $w[1..18]$, compute:

$$\begin{aligned} s &= b + \sum_{i=1}^{18} w_i x_i \\ &= -0.35 + (0.12)(0.90) + (-0.07)(-0.50) + (0.05)(0.30) + (0.09)(0.80) + (-0.11)(-0.70) \\ &\quad + (0.03)(0.60) + (0.08)(0.70) + (-0.02)(0.10) + (0.06)(0.40) + (0.04)(0.50) + (-0.05)(-0.60) \\ &\quad + (0.10)(0.90) + (-0.08)(-0.40) + (0.07)(0.50) + (0.02)(0.20) + (-0.03)(0.10) + (0.11)(0.80) + (-0.06)(-0.50) \\ &= -0.35 + (0.108 + 0.035 + 0.015 + 0.072 + 0.077 + 0.018 + 0.056 - 0.002 + 0.024 + 0.020 + 0.030 \\ &\quad + 0.090 + 0.032 + 0.035 + 0.004 - 0.003 + 0.088 + 0.030) \\ &= -0.35 + 0.729 \\ &= 0.379 \end{aligned}$$

Now compute probability:

$$p = \frac{1}{1 + e^{-s}} = \frac{1}{1 + e^{-0.379}} \approx 0.593632$$

Prediction:

$$\hat{y} = \begin{cases} 1 & \text{if } p \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \Rightarrow \hat{y} = 1$$

Since the true label here is $y = 1$ and $\hat{y} = 1$, you would update counters as:

- PRED_POS increases by 1
- TP increases by 1
- FP, TN, FN do not change for this record

Output (Text; Exact 6 Lines)

Write exactly these lines to output:

```
N <total_records>
PRED_POS <count>
TP <count>
FP <count>
TN <count>
FN <count>
```

Task 2: Sparse Matrix-Vector Multiply (SpMV)

For this task you will use the **SuiteSparse-webbase-1M** sparse matrix. It represents a **web hyperlink graph** in matrix form:

- Each row/column index corresponds to a web page (a node).
- A nonzero entry means a hyperlink relationship between pages.

The file is in **Matrix Market (coordinate)** format:

- Header and comment lines (start with %)
- One size line: **m n nz**, representing $m \times n$ matrix and nz nonzero entries
- Then exactly **nz** lines: **row col value** (Note that it considers 1-based indices i.e., index starting from 1 instead of 0)

A nonzero line: **row col value** represents:

- **row** = **i**: source page i
- **col** = **j**: destination page j
- **value** = $A[i, j]$: link weight from page i to page j (commonly 1.0 for “link exists”)

For example: 7 901 1.0 means page 7 links to page 901 with weight 1.0. If a value is missing in a nonzero line, assume **value** = 1.0.

Task Description

Compute one sparse matrix-vector multiplication:

$$y = A \cdot x$$

Meaning for each row i :

$$y_i = \sum_{(i,j) \in NZ} A_{i,j} \cdot x_j$$

Here x is a score/value per page, and y is the updated score after combining values through the link structure encoded by A .

Fixed Constants

Instead of providing an extra vector file, define x deterministically. For 1-based index $j = 1..n$:

$$x[j] = (j \bmod 1000) / 1000.0$$

Example

If the matrix contains these nonzeros:

```
7 2 1.0
7 100 1.0
7 901 1.0
```

Then:

- $x[2] = 0.002$, $x[100] = 0.100$, $x[901] = 0.901$
- $y[7] = 1.0 \cdot 0.002 + 1.0 \cdot 0.100 + 1.0 \cdot 0.901 = 1.003$

Output (Text)

Writing the full vector y would be extremely large (over one million values) and would make output dominated by file I/O. Therefore you must output a small, comparable signature:

- First line: **M <m> N <n> NZ <nz>**

- Second line: CHECKSUM <value>

Where:

$$\text{CHECKSUM} = \sum_{i=1}^m y_i$$

Print CHECKSUM with at least **6 digits after decimal**.

Task 3: Reachability and Hop-Distance using BFS

You will use the **web-Google** dataset from SNAP. It is a directed web graph:

- Each node represents a web page.
- Each directed edge represents a hyperlink from one page to another.

The dataset is provided as a plain-text **edge list** where:

- Lines beginning with # are comments/metadata; ignore them.
- Every other line contains two integers: $u \ v$

Meaning there is a directed hyperlink (edge) $u \rightarrow v$. For example: 0 824020 means page 0 links to page 824020.

Task Description

Run Breadth-First Search (BFS) from $s = 0$ to compute shortest hop-distance in an unweighted directed graph (0 is the starting page from which hop-distances are measured):

- $\text{dist}[0] = 0$
- When node v is discovered for the first time from a node at distance d , set $\text{dist}[v] = d + 1$
- Nodes never reached must have $\text{dist}[v] = -1$

Then compute:

- REACHABLE: number of nodes with $\text{dist}[v] \neq -1$
- MAXDIST: maximum distance among reachable nodes
- For each level $k = 0, 1, 2, \dots, \text{MAXDIST}$: count nodes where $\text{dist}[v] = k$

Example

Edges:

```
0 3
0 5
3 8
5 8
```

BFS from 0 yields:

- $\text{dist}[0] = 0$
- $\text{dist}[3] = 1, \text{dist}[5] = 1$
- $\text{dist}[8] = 2$

So: REACHABLE = 4, MAXDIST = 2, and level counts are:

```
0 1,
1 2,
2 1.
```

Output (Text)

Write exactly:

```

SOURCE 0
REACHABLE <count>
MAXDIST <d>
0 <count_at_level_0>
1 <count_at_level_1>
...
MAXDIST <count_at_level_MAXDIST>

```

Performance Evaluation and Benchmarking Rules

- Use Standard Time functions like `clock_gettime(CLOCK_MONOTONIC, ...)` for timing instrumentation.
- Test thread counts: $T \in \{1, 2, 4, 8, \dots\}$ up to available logical CPUs (you must include $T = 1$ for all versions).
- Test with different input sizes (at least three variations):
 - **minimal**: first 10% of the dataset
 - **half-dataset**: first 50% of the dataset
 - **full-dataset**: 100% of the dataset

Note: Use a deterministic subset rule (e.g., first N records/edges/nonzeros after skipping headers/comments) so results are reproducible.

- Run **5 trials per configuration**; report **avg(ms)** and **min(ms)**.
- Timed region (must be identical across V1/V2/V3 for the same task):
 - **Task 1 & Task 2**: exclude file I/O (read/parse input first, then time compute only).
 - **Task 3**: time BFS computation; you may exclude graph parsing time if you build adjacency once, but be consistent across V1/V2/V3 and document it in the report.

Results CSVs

Create:

- `results/task1.csv`
- `results/task2.csv`
- `results/task3.csv`

Each row must contain:

```
task,version,threads,chunk,trial,time_ms
```

Use `chunk` as your work-unit size:

- Task 1: records per chunk (for work distribution).
- Task 2: rows (or row-blocks) per chunk (for work distribution).
- Task 3: frontier elements per chunk (or another clearly documented unit).

Example:

```
task1,2,8,200000,3,128.44
```

Report Requirements

Your report must include:

1. Machine specs: CPU model, cores/threads, RAM, OS.
2. Implementation summary per task and version (data structures, decomposition, key synchronization points).

3. Correctness validation strategy.
4. For each task, provide a discussion based on your observed results (not one-liners).
5. You must use at least one profiling/performance analysis tool (e.g., perf, gprof, Intel VTune) and include evidence (screenshots/log excerpts) to support your performance reasoning, explaining what limited speedup (if any), where time was spent, and why the achieved performance matches or deviates from expectations.

Rubric (Marks Breakdown) [Total Marks = 140 Marks]

Criterion	Excellent (9-10)	Good (7-8)	Average (4-6)	Poor (0-3)
A. V1 Serial Implementation (30 Marks) (Each task out of 10)				
Task 1 (Serial)	Implements complete serial pipeline: parse CSV, compute s , p , $\hat{y}$, and all counters; output format exactly as specified; stable results.	Correct serial implementation with minor formatting issues or one minor edge-case issue (e.g., whitespace/printing).	Partially working serial code; missing an important part (e.g., some counters wrong or incomplete parsing).	Incorrect/incomplete serial implementation; wrong outputs or crashes.
Task 2 (Serial)	Correct serial SpMV; correctly builds/uses sparse structure; applies fixed $x[j]$; checksum computed exactly as specified; stable output.	Correct core SpMV but minor format/precision or small edge-case issues.	Partial implementation (e.g., parses but wrong multiply / wrong checksum / inconsistent).	Incorrect/incomplete; crashes or wrong output format.
Task 3 (Serial)	Correct serial BFS from $s = 0$; correct REACHABLE, MAXDIST, and per-level counts; output format exactly as specified.	Mostly correct BFS; minor formatting issue or one small mismatch in level counts.	Partial BFS (some reachable/dist logic works but multiple mismatches).	Incorrect/incomplete BFS; crashes or wrong output.
B. Correctness of V2 (Pthreads, default OS scheduling) (30 Marks) (Each task out of 10)				
Task 1 (V2)	V2 output exactly matches V1 (serial) results; deterministic and stable across repeated runs (no race-related mismatches).	Matches V1 results with minor output formatting issues.	Occasional mismatch with V1 results or unstable output across runs (suggesting correctness issues).	Incorrect V2 results (do not match V1) or crashes.
Task 2 (V2)	Checksum and dimensions match V1 exactly; deterministic and stable across runs; no data races.	Matches V1 with minor formatting/precision output issues.	Checksum occasionally inconsistent / mismatching V1.	Incorrect checksum/output or crashes.
<i>Continued on next page</i>				

Criterion	Excellent	Good	Average	Poor
Task 3 (V2)	All reported values match V1 exactly; REACHABLE, MAXDIST, all levels; deterministic and stable across runs.	Matches V1 with minor formatting issues.	Some values match but multiple level/count mismatches vs V1.	Incorrect BFS output or crashes.

C. Correctness of V3 (Affinity + Scheduling) (30 Marks) (Each task out of 10)

Task 1 (V3)	V3 output exactly matches V1 results; stable across runs; V3 includes explicit CPU affinity for worker threads and a clearly implemented scheduling strategy beyond V2.	Matches V1 with minor formatting issues; affinity/scheduling present but minor issues in clarity/coverage.	Occasional mismatches vs V1 or unstable results across runs; OR V3 features partially/inconsistently applied.	Incorrect V3 results or crashes; OR missing affinity/scheduling requirement.
Task 2 (V3)	Checksum/dimensions exactly match V1; stable across runs; V3 includes explicit CPU affinity and a scheduling strategy beyond V2.	Matches V1 with minor formatting/precision issues; affinity/scheduling present but minor issues.	Partial correctness; checksum mismatches or instability; OR V3 features partially applied.	Incorrect output/checksum or crashes; OR missing affinity/scheduling requirement.
Task 3 (V3)	All BFS outputs match V1 exactly (reachability/-maxdist/levels); stable across runs; V3 includes explicit CPU affinity and correct thread-safe coordination.	Matches V1 with minor formatting issues; affinity/coordination present but minor issues.	Partial correctness; multiple mismatches vs V1; OR V3 features partially applied.	Incorrect BFS output or crashes; OR missing affinity requirement.

D. Performance Evaluation (30 Marks) (3 components, each out of 10)

Continued on next page

Criterion	Excellent	Good	Average	Poor
D1) Thread Scaling (10)	Evaluates $T \in \{1, 2, 4, 8, \dots\}$ up to available logical CPUs for all versions; runs 5 trials/config; reports avg+min; uses a consistent timing region; includes runtime, speedup, and efficiency plots with correct interpretation.	Mostly complete; minor missing thread counts or minor timing inconsistency; plots/interpretation present but slightly weak.	Partial scaling evaluation (missing multiple thread counts/versions or weak plots/interpretation).	Little/no thread-scaling evidence; incorrect methodology or missing baseline $T = 1$.
D2) Input-Size Scaling (10)	Evaluates at least three deterministic input sizes (minimal/half/full) per task; clearly documents the subset rule (e.g., first N records/edges/nonzeros after skipping headers); reports results and discusses trends across sizes.	Has all three sizes but minor issues (subset rule unclear, or missing a few configurations).	Limited size variation (e.g., only two sizes) or weak discussion of trends across sizes.	No meaningful input-size variation or non-reproducible subsets.
D3) Profiling-Based Performance Reasoning (10)	Uses at least one profiling tool (e.g., perf , gprof , Intel VTune); includes evidence (screenshots/logs); discussion references concrete tool metrics (e.g., hotspots, cache-miss rate, branch mispredicts, CPI, context switches, lock contention) to explain achieved vs expected performance.	Tool used with evidence, but analysis is shallow or metrics are reported without strong linkage to conclusions.	Tool usage incomplete (little evidence) or reasoning mostly generic with minimal metrics.	No profiling tool used and/or no evidence provided, or reasoning is purely speculative.

E. Report Quality & Reproducibility (20 Marks) (2 components, each out of 10)

Continued on next page

Criterion	Excellent	Good	Average	Poor
E1) Report Quality (10)	Well-structured report; clear writing; includes machine specs; explains approach per task/version; presents results with readable tables/plots; discussion is task-wise and evidence-based (not one-liners).	Mostly clear and structured; minor missing details (e.g., limited explanation for one task/version) or minor presentation issues in plots/tables.	Report lacks important explanations (multiple missing details) or results presentation is hard to follow; discussion is shallow.	Weak/incomplete report or major missing sections; unclear/incorrect presentation of results.
E2) Reproducibility & Packaging (10)	Submission is easy to run: consistent CLI for all tasks/versions; correct output formats; clear README with steps/commands; Makefile works; no hardcoded paths; results CSVs complete and consistent with stated experiments.	Mostly reproducible; minor issues (e.g., small README gaps, minor path assumptions, or occasional inconsistency in filenames/CSVs).	Partially reproducible; missing important steps/files (e.g., incomplete README/Makefile/CSVs) or inconsistent CLI/output conventions.	Not reproducible; missing critical files/instructions or broken build/run process.

Good luck with your first assignment! This exercise will give you a solid foundation in understanding how parallelism can optimize performance, a concept that will be crucial as we progress in this course.